



Faculty of Engineering and Technology
Electrical and Computer Engineering Department

ENCS4370

**Design and Verification of a Simplified
Predicated RISC Processor using Verilog**

Project #2

Group members:

Asma'a Abdalrahman Fares	1210084
Aya Abdalrahman Fares	1222654
Baker Mustafa Shalabi	1231050

Instructor: Dr. Ahmad Afaneh

Section: 2

Date: 8-JAN-2026

Contents

1	Introduction	4
1.1	Project Objectives	4
1.2	Processor Overview	4
2	Processor Specifications	4
2.1	Register File	4
2.2	Memory Organization	4
2.3	Instruction Formats	5
2.3.1	R-Type (Register Type)	5
2.3.2	I-Type (Immediate Type)	5
2.3.3	J-Type (Jump Type)	5
3	Instruction Set Architecture (ISA)	5
3.1	Instruction Encoding Table	5
3.2	Predicated Execution	5
4	Control Signal Generation	6
4.1	Control Signals Overview	6
4.2	Control Truth Table	6
4.3	Boolean Equations	6
4.4	ALU Control Logic	11
5	Datapath Design	11
5.1	Pipeline Architecture Overview	11
5.2	Pipeline Registers	11
5.3	Complete Datapath Diagram	12
5.4	Major Components	12
5.4.1	Program Counter (PC)	12
5.4.2	Instruction Memory	13
5.4.3	Register File	13
5.4.4	ALU (Arithmetic Logic Unit)	13
5.4.5	Data Memory	13
6	Control Path Design	14
6.1	Control Unit Architecture	14
6.2	Predicate Logic	14
6.3	State Diagram	15
7	State Diagram	15
8	Hazard Handling	15
8.1	Data Hazards	15
8.1.1	Forwarding (Data Bypassing)	15
8.1.2	Load-Use Hazard Detection	16
8.2	Control Hazards	16
8.2.1	Branch/Jump Handling	16
9	Implementation Details	17

9.1	Design Choices and Justifications	17
9.1.1	Pipeline Architecture	17
9.1.2	Forwarding Implementation	17
9.1.3	Load-Use Hazard	17
9.1.4	Branch Resolution	17
9.1.5	Predicate Register Read	17
9.1.6	Word Addressing	17
9.2	Special Register Handling	17
9.3	Immediate Value Extension	18
10	Verification and Testing	18
10.1	Testbench Architecture	18
10.1.1	Task-Based Verification Structure	18
10.2	Test Programs	19
10.2.1	Test 1: Logical Operations (test_logical_rtype.hex)	19
10.2.2	Test 2: Load-Use Hazard (test_load_use_hazard.hex)	20
10.2.3	Test 3: Memory with Offset (test_memory_offset.hex)	20
10.2.4	Test 4: Jump Instruction (test_jump.hex)	21
10.2.5	Test 5: CALL and JR (test_call.hex)	21
10.2.6	Test 6: Predicated Execution (test_predicated_memory.hex)	22
10.2.7	Test 7: Consecutive Hazards (test_consecutive_hazards.hex)	23
10.3	Simulation Results	23
10.3.1	Test 1: Logical R-Type Instructions	24
10.3.2	Test 2: Load-Use Hazard Handling	24
10.3.3	Test 3: Consecutive Data Hazards	25
10.3.4	Test 4: Jump Instruction	25
10.3.5	Test 5: CALL Instruction	26
10.3.6	Test 6: Memory Access with Offset	26
10.3.7	Test 7: Predicated Memory Instructions	27
10.4	Test Results Summary	28
11	Performance Analysis	28
12	Code Organization	28
12.1	File Structure	28
13	Team Member Contributions	29
13.1	Work Distribution	29
14	Challenges and Solutions	30
14.1	Challenge 1: Load-Use Hazard Detection	30
14.2	Challenge 2: Predicate Register Forwarding	30
14.3	Challenge 3: CALL Instruction Return Address	30
14.4	Challenge 4: Special Register Management	30
15	Conclusion	31
15.1	Key Achievements	31
15.2	Learning Outcomes	31

16 AI Tools Disclosure

31

1 Introduction

This report presents the design, implementation, and verification of a 32-bit predicated RISC processor. The processor features a 5-stage pipeline architecture and supports predicated execution, allowing conditional instruction execution based on predicate registers.

1.1 Project Objectives

- Design a complete datapath and control path for a predicated RISC processor
- Implement the processor in System Verilog
- Verify functionality through comprehensive simulation
- Support 15 instructions including arithmetic, logical, memory, and control flow operations

1.2 Processor Overview

The processor includes:

- 32 general-purpose 32-bit registers (R0-R31)
- Separate instruction and data memories (word-addressable)
- 5-stage pipeline: IF, ID, EX, MEM, WB
- Predicated execution support
- Hardware forwarding and hazard detection

2 Processor Specifications

2.1 Register File

- **R0**: Hardwired to zero (constant 0)
- **R1-R29**: General-purpose registers
- **R30**: Program Counter (PC)
- **R31**: Return address register (for CALL instruction)

2.2 Memory Organization

- **Instruction Memory**: 1024 words $\times$ 32 bits
- **Data Memory**: 1024 words $\times$ 32 bits
- **Addressing**: Word-addressable (each address points to a 32-bit word)

2.3 Instruction Formats

2.3.1 R-Type (Register Type)

Opcode [31:27]	Rp [26:22]	Rd [21:17]	Rs [16:12]	Rt [11:7]	Unused [6:0]
5 bits	5 bits	5 bits	5 bits	5 bits	7 bits

2.3.2 I-Type (Immediate Type)

Opcode [31:27]	Rp [26:22]	Rd [21:17]	Rs [16:12]	Immediate [11:0]
5 bits	5 bits	5 bits	5 bits	12 bits

2.3.3 J-Type (Jump Type)

Opcode [31:27]	Rp [26:22]	Offset [21:0]
5 bits	5 bits	22 bits

3 Instruction Set Architecture (ISA)

3.1 Instruction Encoding Table

Instruction	Type	Opcode	RTN
ADD Rd, Rs, Rt, Rp	R	0	If ($Rp \neq 0$) then $Rd = Rs + Rt$
SUB Rd, Rs, Rt, Rp	R	1	If ($Rp \neq 0$) then $Rd = Rs - Rt$
OR Rd, Rs, Rt, Rp	R	2	If ($Rp \neq 0$) then $Rd = Rs Rt$
NOR Rd, Rs, Rt, Rp	R	3	If ($Rp \neq 0$) then $Rd = \sim (Rs Rt)$
AND Rd, Rs, Rt, Rp	R	4	If ($Rp \neq 0$) then $Rd = Rs \& Rt$
ADDI Rd, Rs, Imm, Rp	I	5	If ($Rp \neq 0$) then $Rd = Rs + \text{SignExt}(\text{Imm})$
ORI Rd, Rs, Imm, Rp	I	6	If ($Rp \neq 0$) then $Rd = Rs \text{ZeroExt}(\text{Imm})$
NORI Rd, Rs, Imm, Rp	I	7	If ($Rp \neq 0$) then $Rd = \sim (Rs \text{ZeroExt}(\text{Imm}))$
ANDI Rd, Rs, Imm, Rp	I	8	If ($Rp \neq 0$) then $Rd = Rs \& \text{ZeroExt}(\text{Imm})$
LW Rd, Imm(Rs), Rp	I	9	If ($Rp \neq 0$) then $Rd = \text{Mem}[Rs + \text{SignExt}(\text{Imm})]$
SW Rd, Imm(Rs), Rp	I	10	If ($Rp \neq 0$) then $\text{Mem}[Rs + \text{SignExt}(\text{Imm})] = Rd$
J Label, Rp	J	11	If ($Rp \neq 0$) then $PC = PC + \text{SignExt}(\text{Offset})$
CALL Label, Rp	J	12	If ($Rp \neq 0$) then $R31 = PC + 1$, $PC = PC + \text{SignExt}(\text{Offset})$
JR Rs, Rp	R	13	If ($Rp \neq 0$) then $PC = Rs$

Table 1: Complete Instruction Set (Corrected Opcodes - Bold indicates corrections from original spec)

3.2 Predicated Execution

- Every instruction includes a predicate register field (Rp)
- If $Rp = R0$, instruction executes unconditionally

- If $Rp \neq R0$ and content is zero, instruction is nullified (no operation)
- If $Rp \neq R0$ and content is non-zero, instruction executes normally

4 Control Signal Generation

4.1 Control Signals Overview

The control unit generates the following signals based on the instruction opcode:

Signal	Description
RegWrite	Enable register file write
MemRead	Enable data memory read
MemWrite	Enable data memory write
MemToReg	Select memory data for register writeback
ALUSrc	Select ALU operand: 0=Register, 1=Immediate
ALUOp[3:0]	ALU operation selection
IsJump	Indicates jump instruction
IsCall	Indicates call instruction
IsJR	Indicates jump register instruction

Table 2: Control Signals

4.2 Control Truth Table

Opcode	Instr	RegW	MemR	MemW	Mem2R	ALUSrc	ALUOp	Jump	Call	JR
0	ADD	1	0	0	0	0	0000	0	0	0
1	SUB	1	0	0	0	0	0001	0	0	0
2	OR	1	0	0	0	0	0010	0	0	0
3	NOR	1	0	0	0	0	0011	0	0	0
4	AND	1	0	0	0	0	0100	0	0	0
5	ADDI	1	0	0	0	1	0000	0	0	0
6	ORI	1	0	0	0	1	0010	0	0	0
7	NORI	1	0	0	0	1	0011	0	0	0
9	ANDI	1	0	0	0	1	0100	0	0	0
10	LW	1	1	0	1	1	0000	0	0	0
11	SW	0	0	1	0	1	0000	0	0	0
12	J	0	0	0	0	X	XXXX	1	0	0
13	CALL	1	0	0	0	X	XXXX	0	1	0
14	JR	0	0	0	0	X	XXXX	0	0	1

Table 3: Complete Control Truth Table

4.3 Boolean Equations

The control signals are generated using combinational logic derived from the instruction opcode field Op[4 : 0]. Boolean expressions were obtained by constructing Karnaugh maps for each control signal based on the control truth table.

Karnaugh Map for RegWrite (Grouped)

		Op_4Op_3			
		00	01	11	10
$Op_2Op_1Op_0$	00	1	1	1	1
	01	1	1	1	1
	11	1	0	0	0
	10	1	1	0	0

Karnaugh Map for MemRead (Grouped)

		Op_4Op_3			
		00	01	11	10
$Op_2Op_1Op_0$	00	0	0	0	0
	01	0	0	0	0
	11	0	0	0	0
	10	0	1	0	0

Karnaugh Map for MemWrite (Grouped)

		Op_4Op_3			
		00	01	11	10
$Op_2Op_1Op_0$	00	0	0	0	0
	01	0	0	0	0
	11	0	0	0	0
	10	0	0	0	1

Karnaugh Map for MemToReg (Grouped)

		Op_4Op_3			
		00	01	11	10
$Op_2Op_1Op_0$	00	0	0	0	0
	01	0	0	0	0
	11	0	0	0	0
	10	0	1	0	0

Karnaugh Map for ALUSrc (Grouped)

		Op_4Op_3			
		00	01	11	10
$Op_2Op_1Op_0$	00	0	0	0	0
	01	0	1	1	1
	11	0	0	0	0
	10	1	1	0	1

Karnaugh Map for Jump (Grouped)

		Op_4Op_3			
		00	01	11	10
$Op_2Op_1Op_0$	00	0	0	0	0
	01	0	0	0	0
	11	0	0	0	0
	10	0	0	1	0

Karnaugh Map for Call (Grouped)

		Op_4Op_3			
		00	01	11	10
$Op_2Op_1Op_0$	00	0	0	0	0
	01	0	0	0	0
	11	1	0	0	0
	10	0	0	0	0

Karnaugh Map for JR (Grouped)

		Op_4Op_3			
		00	01	11	10
$Op_2Op_1Op_0$	00	0	0	0	0
	01	0	0	0	0
	11	0	1	0	0
	10	0	0	0	0

Predicate Gating

$$\text{RegWrite}_{final} = \text{RegWrite} \cdot \text{Predicate_Pass}$$

$$\text{MemWrite}_{final} = \text{MemWrite} \cdot \text{Predicate_Pass}$$

For clarity, the following intermediate instruction-type signals are defined:

$$R_type = \overline{Op_4} \overline{Op_3} \overline{Op_2} \overline{Op_1} + \overline{Op_4} \overline{Op_3} \overline{Op_2} Op_1 + \overline{Op_4} \overline{Op_3} Op_2 \overline{Op_1} + \overline{Op_4} \overline{Op_3} Op_2 Op_1$$

$$I_type = \overline{Op_4} Op_3 + Op_4 \overline{Op_3}$$

$$J_type = Op_4 Op_3$$

These correspond to arithmetic/logical R-type instructions, immediate-type instructions, and jump-type instructions respectively.

Main Control Signals

$$\text{RegWrite} = R_type + I_type + \text{CALL}$$

$$\text{MemRead} = \text{LW}$$

$$\text{MemWrite} = \text{SW}$$

$$\text{MemToReg} = \text{LW}$$

$$\text{ALUSrc} = \text{ADDI} + \text{ORI} + \text{NORI} + \text{ANDI} + \text{LW} + \text{SW}$$

$$\text{IsJump} = \text{J}$$

$$\text{IsCall} = \text{CALL}$$

$$\text{IsJR} = \text{JR}$$

Opcode-Specific Decode Signals

Each instruction-specific signal is asserted when its corresponding opcode pattern is detected:

$$\text{ADD} = \overline{Op_4} \overline{Op_3} \overline{Op_2} \overline{Op_1} \overline{Op_0}$$

$$\text{SUB} = \overline{Op_4} \overline{Op_3} \overline{Op_2} \overline{Op_1} Op_0$$

$$\text{OR} = \overline{Op_4} \overline{Op_3} \overline{Op_2} Op_1 \overline{Op_0}$$

$$\text{NOR} = \overline{Op_4} \overline{Op_3} \overline{Op_2} Op_1 Op_0$$

$$\text{AND} = \overline{Op_4} \overline{Op_3} Op_2 \overline{Op_1} \overline{Op_0}$$

$$\text{ADDI} = \overline{Op_4} Op_3 \overline{Op_2} \overline{Op_1} \overline{Op_0}$$

$$\text{ORI} = \overline{Op_4} Op_3 \overline{Op_2} \overline{Op_1} Op_0$$

$$\text{NORI} = \overline{Op_4} Op_3 \overline{Op_2} Op_1 \overline{Op_0}$$

$$\text{ANDI} = \overline{Op_4} Op_3 \overline{Op_2} Op_1 Op_0$$

$$\text{LW} = \overline{Op_4} Op_3 Op_2 \overline{Op_1} \overline{Op_0}$$

$$\text{SW} = \overline{Op_4} Op_3 Op_2 \overline{Op_1} Op_0$$

$$\text{J} = Op_4 Op_3 \overline{Op_2} \overline{Op_1} \overline{Op_0}$$

$$\text{CALL} = Op_4 Op_3 \overline{Op_2} \overline{Op_1} Op_0$$

$$\text{JR} = Op_4 Op_3 \overline{Op_2} Op_1 \overline{Op_0}$$

Predicate Interaction

All write-enable and side-effect signals are finally gated by the predicate evaluation result:

$$\text{RegWrite}_{final} = \text{RegWrite} \cdot \text{Predicate_Pass}$$

$$\text{MemWrite}_{final} = \text{MemWrite} \cdot \text{Predicate_Pass}$$

This ensures that instructions with a false predicate do not modify architectural state, while still flowing through the pipeline without stalling.

4.4 ALU Control Logic

ALUOp[3:0]	Operation	Result
0000	ADD	$A + B$
0001	SUB	$A - B$
0010	OR	$A \mid B$
0011	NOR	$\sim(A \mid B)$
0100	AND	$A \& B$

Table 4: ALU Operations

5 Datapath Design

5.1 Pipeline Architecture Overview

The processor implements a 5-stage pipeline:

1. **IF (Instruction Fetch)**: Fetch instruction from instruction memory
2. **ID (Instruction Decode)**: Decode instruction and read registers
3. **EX (Execute)**: Perform ALU operations and predicate evaluation
4. **MEM (Memory Access)**: Access data memory for load/store
5. **WB (Write Back)**: Write result back to register file

5.2 Pipeline Registers

IF/ID Register:

- Instruction [31:0]
- PC [31:0]
- Valid bit

ID/EX Register:

- PC, Rs data, Rt data, Rp data, Rd data
- Immediate value, Rd, Rs, Rt, Rp, Opcode

- Control signals: RegWrite, MemRead, MemWrite, MemToReg, ALUSrc, ALUOp, IsJump, IsCall, IsJR

- Valid bit

EX/MEM Register:

- ALU result, Write data, PC
- Rd address
- Control signals: RegWrite, MemRead, MemWrite, MemToReg
- Predicate pass flag
- Valid bit

MEM/WB Register:

- ALU result, Memory data
- Rd address
- Control signals: RegWrite, MemToReg
- Valid bit

5.3 Complete Datapath Diagram

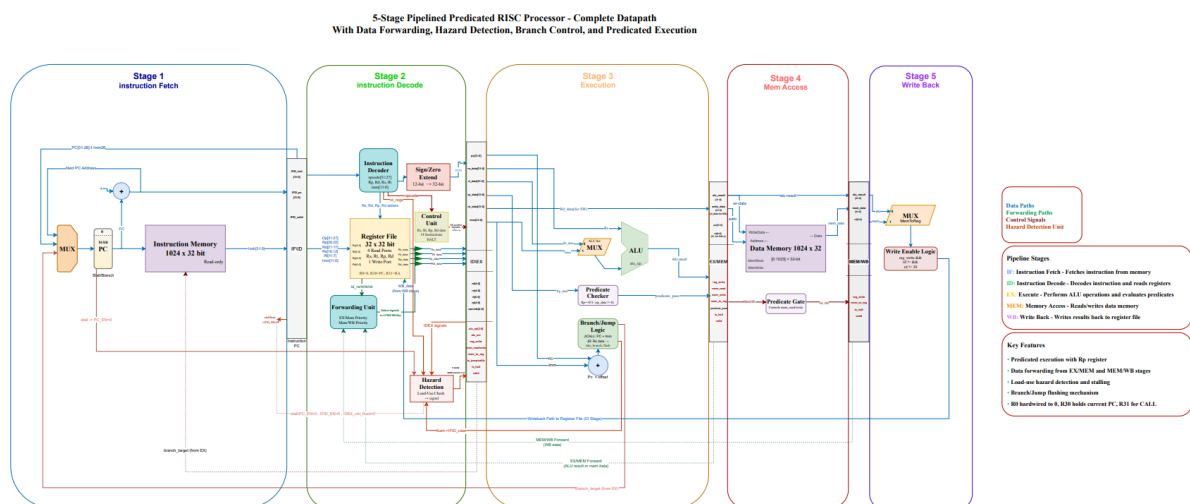


Figure 1: Complete Processor Datapath

5.4 Major Components

5.4.1 Program Counter (PC)

- 32-bit register
- Increments by 1 each cycle (word addressing)
- Updated by branch/jump instructions
- Connected to R30

5.4.2 Instruction Memory

- 1024×32 -bit ROM
- Word-addressable
- Asynchronous read
- Initialized from hex file

5.4.3 Register File

- 32 registers $\times$ 32 bits
- 4 read ports (Rs, Rt, Rp, Rd)
- 1 write port
- Synchronous write, asynchronous read
- Special registers: R0=0, R30=PC, R31=return address

5.4.4 ALU (Arithmetic Logic Unit)

- 32-bit operands
- Operations: ADD, SUB, OR, NOR, AND
- Two inputs: register or immediate
- One 32-bit output

5.4.5 Data Memory

- 1024×32 -bit RAM
- Word-addressable
- Synchronous write, asynchronous read
- Supports LW/SW operations

6 Control Path Design

6.1 Control Unit Architecture

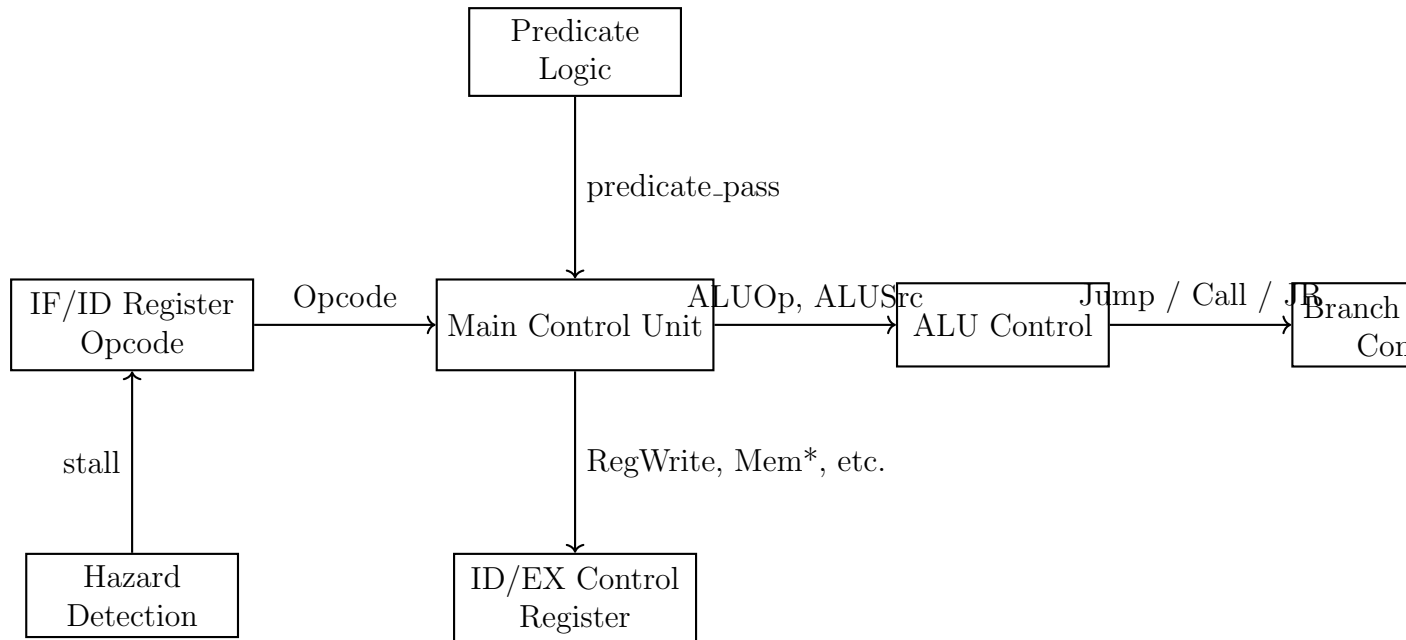


Figure 2: Control Path Diagram of the Pipelined Predicated RISC Processor

The control unit is implemented as combinational logic that decodes the opcode field and generates appropriate control signals.

6.2 Predicate Logic

The predicate evaluation is performed in the EX stage:

Listing 1: Predicate Logic Implementation

```

1 always_comb begin
2     if (IDEX_rp == 5'd0)
3         ex_predicate_pass = 1; // R0 predicate = always execute
4     else
5         ex_predicate_pass = (IDEX_rp_data != 0); // Execute if
6                                     Rp != 0
7 end
    
```

Instructions with failed predicates are nullified by:

- Preventing register writes (RegWrite & predicate_pass)
- Preventing memory writes (MemWrite & predicate_pass)
- Preventing PC updates for branches

6.3 State Diagram

7 State Diagram

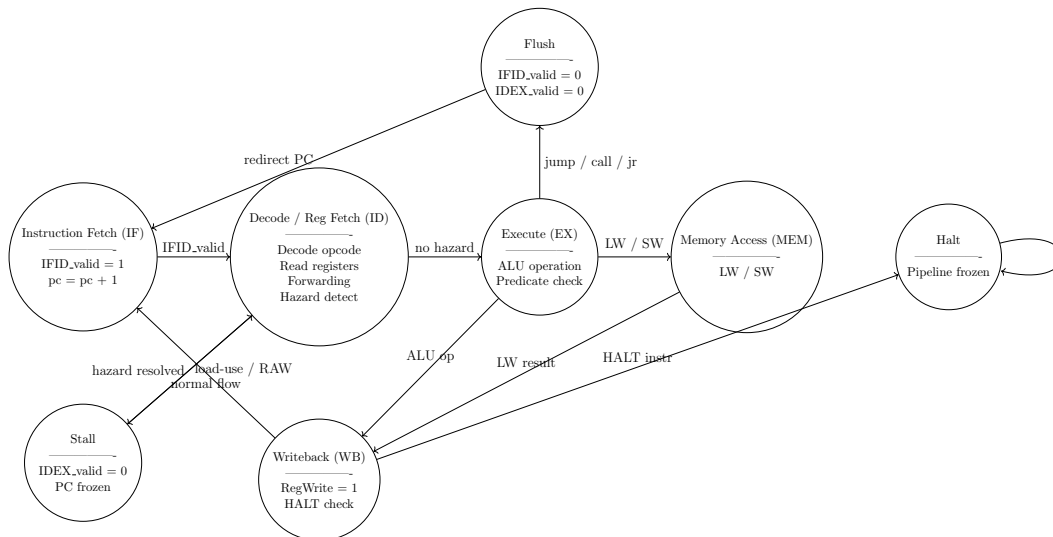


Figure 3: RTL-Derived Control FSM for Pipelined Predicated RISC Processor

8 Hazard Handling

8.1 Data Hazards

8.1.1 Forwarding (Data Bypassing)

The processor implements full forwarding from:

- **EX/MEM** → **ID**: Forwards ALU results or loaded data
- **MEM/WB** → **ID**: Forwards writeback data

Forwarding priority (highest first):

1. EX/MEM stage (most recent)
2. MEM/WB stage
3. Register file

Listing 2: Forwarding Logic Example

```

1 // Forward from EX/MEM (highest priority)
2 if (EXMEM_reg_write && EXMEM_valid && EXMEM_predicate_pass
3     && EXMEM_rd != 0 && EXMEM_rd == id_rs)
4     id_rs_forward = EXMEM_alu_result;
5
6 // Forward from MEM/WB (lower priority)
7 else if (MEMWB_reg_write && MEMWB_valid && MEMWB_rd != 0
8         && MEMWB_rd == id_rs)
9     id_rs_forward = wb_write_data;
    
```


8.1.2 Load-Use Hazard Detection

A pipeline stall is required when:

- An instruction in ID stage depends on a load in EX stage
- The load data is not yet available

Listing 3: Hazard Detection

```

1 always_comb begin
2     stall = 0;
3     if (IDEX_mem_read && IDEX_valid && IDEX_rd != 0) begin
4         if ((IDEX_rd == id_rs) || (IDEX_rd == id_rt) ||
5             (IDEX_rd == id_rp) || (IDEX_rd == id_rd))
6             stall = 1; // Insert bubble
7     end
8 end
    
```

8.2 Control Hazards

8.2.1 Branch/Jump Handling

Branch target is calculated in EX stage, requiring:

- **Flush:** Invalidate IF/ID and ID/EX stages
- **PC Update:** Load branch target
- 2-cycle branch penalty

Listing 4: Branch Resolution

```

1 always_comb begin
2     take_branch = 0;
3     flush = 0;
4
5     if (IDEX_valid && ex_predicate_pass) begin
6         if (IDEX_is_jump || IDEX_is_call) begin
7             branch_target = IDEX_pc + IDEX_imm;
8             take_branch = 1;
9             flush = 1;
10        end else if (IDEX_is_jr) begin
11            branch_target = IDEX_rs_data;
12            take_branch = 1;
13            flush = 1;
14        end
15    end
16 end
    
```

9 Implementation Details

9.1 Design Choices and Justifications

9.1.1 Pipeline Architecture

Choice: 5-stage pipeline

Justification: Standard RISC design balancing complexity and performance. Allows multiple instructions in flight while keeping control logic manageable.

9.1.2 Forwarding Implementation

Choice: Full forwarding with priority-based selection

Justification: Minimizes stalls for most data hazards. EX/MEM has priority as it contains the most recent data.

9.1.3 Load-Use Hazard

Choice: Mandatory 1-cycle stall

Justification: Memory data not available until MEM stage, making forwarding impossible. Stalling is simpler than advanced scheduling.

9.1.4 Branch Resolution

Choice: Resolve in EX stage

Justification: Allows predicate evaluation before branch decision. Accepts 2-cycle penalty for simpler control logic.

9.1.5 Predicate Register Read

Choice: Four read ports on register file (Rs, Rt, Rp, Rd)

Justification: Allows simultaneous reading of predicate and operands. Rd read needed for SW instruction.

9.1.6 Word Addressing

Choice: PC increments by 1, not 4

Justification: Simplifies addressing logic as specified. Each address points to one 32-bit word.

9.2 Special Register Handling

- **R0:** Always forced to 0 in sequential block
- **R30:** Updated with PC value each cycle
- **R31:** Written by CALL instruction with PC+1

9.3 Immediate Value Extension

- **Sign Extension:** ADDI, LW, SW, J, CALL (arithmetic/addresses)
- **Zero Extension:** ORI, NORI, ANDI (logical operations)

10 Verification and Testing

10.1 Testbench Architecture

The verification environment is implemented using a self-checking, task-based SystemVerilog testbench. The testbench is responsible for stimulus generation, program loading, execution control, and result verification of the pipelined predicated RISC processor.

The testbench instantiates the processor as a device under test (DUT) and directly accesses internal signals and memories for detailed visibility during simulation. This allows accurate observation of pipeline behavior, hazard handling, and predicated execution.

Clock generation is performed using an always-running process that produces a 10 ns clock period (100 MHz). A synchronous reset sequence is applied at the beginning of each test to ensure a clean processor state before program execution.

The testbench:

- Instantiates the processor module
- Generates clock signal (period = 10ns)
- Applies reset pulse
- Loads test programs from hex files
- Monitors register file and memory contents
- Checks for halt condition
- Displays final results

10.1.1 Task-Based Verification Structure

To improve modularity and readability, the testbench is organized around reusable SystemVerilog tasks. Each task encapsulates a specific verification function, allowing tests to be written in a structured and scalable manner.

The main verification tasks include:

- **load_program:** Loads instruction memory contents from external hexadecimal files using `$readmemh`
- **reset_processor:** Applies a controlled reset sequence and initializes cycle counters
- **run_until_halt:** Executes the processor until a HALT instruction is detected or a timeout occurs
- **display_pipeline_state:** Prints the contents of all pipeline registers (IF/ID, ID/EX, EX/MEM, MEM/WB) every cycle

- `display_registers`: Displays the full register file contents at the end of each test
- `display_memory`: Displays selected data memory locations
- `check_register` and `check_memory`: Perform self-checking comparisons between actual and expected values

10.2 Test Programs

Simulation Results Summary All test programs were executed using the self-checking SystemVerilog testbench. For each test, pipeline activity was monitored cycle-by-cycle, and final register and memory contents were automatically verified against expected values. All tests completed successfully with no mismatches, confirming correct functionality of the datapath, control logic, hazard handling, and predicated execution.

10.2.1 Test 1: Logical Operations (`test_logical_rtype.hex`)

Objective: Verify OR, AND, NOR operations

Listing 5: Assembly Code

```

1 ADDI R1, R0, 0xAA, R0 // R1 = 0xAA = 10101010
2 ADDI R2, R0, 0x55, R0 // R2 = 0x55 = 01010101
3 OR   R3, R1, R2, R0   // R3 = 0xFF = 11111111
4 AND  R4, R1, R2, R0   // R4 = 0x00 = 00000000
5 NOR  R5, R1, R2, R0   // R5 = 0xFFFFF00
6 HALT
    
```

Expected Results:

- R1 = 0x000000AA
- R2 = 0x00000055
- R3 = 0x000000FF (OR result)
- R4 = 0x00000000 (AND result)
- R5 = 0xFFFFF00 (NOR result)

Hex Code:

```

280200AA // PC 0: ADDI R1, R0, 0xAA
28040055 // PC 1: ADDI R2, R0, 0x55
10061100 // PC 2: OR   R3, R1, R2
20081100 // PC 3: AND  R4, R1, R2
180A1100 // PC 4: NOR  R5, R1, R2
70000000 // PC 5: HALT
    
```

10.2.2 Test 2: Load-Use Hazard (test_load_use_hazard.hex)

Objective: Verify hazard detection and stalling

Listing 6: Assembly Code

```

1 ADDI R1, R0, 10, R0    // R1 = 10 (address)
2 ADDI R2, R0, 42, R0    // R2 = 42 (data)
3 SW R2, R1, 0, R0       // Mem[10] = 42
4 LW R3, R1, 0, R0       // R3 = Mem[10] = 42
5 ADDI R4, R3, 8, R0     // R4 = 50 (tests load-use hazard)
6 HALT
    
```

Expected Behavior:

- Pipeline stalls 1 cycle between LW and ADDI
- R3 = 42 (0x0000002A)
- R4 = 50 (0x00000032)
- Mem[10] = 42

Hex Code:

```

2802000A // PC 0: ADDI R1, R0, 10
2804002A // PC 1: ADDI R2, R0, 42
50041000 // PC 2: SW R2, R1, 0 → Mem[10] = 42
48061000 // PC 3: LW R3, R1, 0 → R3 = Mem[10] = 42
28083008 // PC 4: ADDI R4, R3, 8 → R4 = 50
70000000 // PC 5: HALT
    
```

10.2.3 Test 3: Memory with Offset (test_memory_offset.hex)

Objective: Verify immediate offset calculation in memory operations

Listing 7: Assembly Code

```

1 ADDI R1, R0, 10, R0    // R1 = 10 (base)
2 ADDI R2, R0, 100, R0   // R2 = 100
3 ADDI R3, R0, 200, R0   // R3 = 200
4 SW R2, R1, 0, R0       // Mem[10+0] = 100
5 SW R3, R1, 5, R0       // Mem[10+5] = 200
6 LW R4, R1, 0, R0       // R4 = Mem[10] = 100
7 LW R5, R1, 5, R0       // R5 = Mem[15] = 200
8 HALT
    
```

Expected Results:

- Mem[10] = 100 (0x00000064)
- Mem[15] = 200 (0x000000C8)
- R4 = 100
- R5 = 200

Hex Code:

```

2802000A // PC 0: ADDI R1, R0, 10 → R1 = 10 (base address for memory operations)

28040064 // PC 1: ADDI R2, R0, 100 → R2 = 100 (first data value to store)

280600C8 // PC 2: ADDI R3, R0, 200 → R3 = 200 (second data value to store)

50041000 // PC 3: SW R2, R1, 0 → Mem[10+0] = Mem[10] = 100 (store with offset=0)

50061005 // PC 4: SW R3, R1, 5 → Mem[10+5] = Mem[15] = 200 (store with offset=5, test)

48081000 // PC 5: LW R4, R1, 0 → R4 = Mem[10+0] = 100 (load with offset=0, verifies)

480A1005 // PC 6: LW R5, R1, 5 → R5 = Mem[10+5] = 200 (load with offset=5, verifies)

70000000 // PC 7: HALT → Stop execution

```

10.2.4 Test 4: Jump Instruction (test_jump.hex)

Objective: Verify unconditional jump

Listing 8: Assembly Code

1	ADDI R1, R0, 0, R0	// PC 0: R1 = 0
2	J +3, R0	// PC 1: Jump to PC 4
3	ADDI R1, R0, 99, R0	// PC 2: SKIPPED
4	ADDI R1, R0, 99, R0	// PC 3: SKIPPED
5	ADDI R2, R0, 20, R0	// PC 4: R2 = 20
6	HALT	// PC 5

Expected Results:

- R1 = 0 (instructions at PC 2, 3 skipped)
- R2 = 20 (0x00000014)
- PC jumps from 1 to 4

Hex Code:

```

28020000 // PC 0: ADDI R1, R0, 0           ; initialize R1 = 0
58000003 // PC 1: J +3                     ; jump to PC = 4
28020063 // PC 2: ADDI R1, R0, 99          ; skipped due to jump
28020063 // PC 3: ADDI R1, R0, 99          ; skipped due to jump
28040014 // PC 4: ADDI R2, R0, 20          ; executed after jump
70000000 // PC 5: HALT                     ; stop execution

```

10.2.5 Test 5: CALL and JR (test_call.hex)

Objective: Verify function call and return

Listing 9: Assembly Code

```

1 ADDI R1, R0, 5, R0    // PC 0: R1 = 5
2 CALL +3, R0          // PC 1: Jump to PC 4, R31 = 2
3 ADDI R2, R0, 20, R0   // PC 2: R2 = 20 (after return)
4 HALT                 // PC 3
5 ADDI R3, R0, 50, R0   // PC 4: R3 = 50 (function)
6 JR R31, R0           // PC 5: Return to PC 2

```

Expected Results:

- R1 = 5
- R31 = 2 (return address saved by CALL)
- R3 = 50 (function executed)
- R2 = 20 (return successful)

Hex Code:

```

28020005 // PC 0: ADDI R1, R0, 5
60000003 // PC 1: CALL +3 (to PC 4, saves R31=2)
28040014 // PC 2: ADDI R2, R0, 20
70000000 // PC 3: HALT
28060032 // PC 4: ADDI R3, R0, 50 (function)
6801F000 // PC 5: JR R31 (return to PC 2)

```

10.2.6 Test 6: Predicated Execution (test_predicated_memory.hex)

Objective: Verify predicate register functionality

Listing 10: Assembly Code

```

1 ADDI R1, R0, 1, R0    // PC 0: R1 = 1 (true predicate)
2 ADDI R2, R0, 0, R0    // PC 1: R2 = 0 (false predicate)
3 ADDI R3, R0, 100, R0  // PC 2: R3 = 100
4 ADDI R4, R0, 10, R0   // PC 3: R4 = 10
5 [R1] ADDI R4, R0, 50, R0 // PC 4: R4 = 50 (EXECUTES)
6 [R2] SW R5, R4, 0, R0  // PC 5: (NOT EXECUTED)
7 [R1] LW R2, R0, 0, R0  // PC 6: R2 = Mem[0] (EXECUTES)
8 HALT                 // PC 7

```

Expected Results:

- R4 = 50 (predicated ADDI executed with R1=1)
- Mem[50] unchanged (SW not executed with R2=0)
- R2 = Mem[0] (predicated LW executed)

Hex Code:

```

28020001 // PC 0: ADDI R1, R0, 1
28040000 // PC 1: ADDI R2, R0, 0
28060064 // PC 2: ADDI R3, R0, 100
2808000A // PC 3: ADDI R4, R0, 10
28480032 // PC 4: [R1] ADDI R4, R0, 50
508A4000 // PC 5: [R2] SW R5, R4, 0
48440000 // PC 6: [R1] LW R2, R0, 0
70000000 // PC 7: HALT
    
```

10.2.7 Test 7: Consecutive Hazards (test_consecutive_hazards.hex)

Objective: Verify forwarding with back-to-back dependencies

Listing 11: Assembly Code

1	ADDI R1, R0, 5, R0	// R1 = 5
2	ADDI R2, R1, 3, R0	// R2 = R1 + 3 = 8 (RAW hazard)
3	ADDI R3, R2, 3, R0	// R3 = R2 + 3 = 11 (RAW hazard)
4	ADDI R4, R3, 3, R0	// R4 = R3 + 3 = 14 (RAW hazard)
5	ADDI R5, R4, 3, R0	// R5 = R4 + 3 = 17 (RAW hazard)
6	HALT	

Expected Results:

- R1 = 5
- R2 = 8
- R3 = 11
- R4 = 14
- R5 = 17
- Tests forwarding through multiple stages

Hex Code:

```

28020005 // PC 0: ADDI R1, R0, 5          ; R1 = 5
28041003 // PC 1: ADDI R2, R1, 3          ; RAW hazard on R1
28062003 // PC 2: ADDI R3, R2, 3          ; RAW hazard on R2
28083003 // PC 3: ADDI R4, R3, 3          ; RAW hazard on R3
280A4003 // PC 4: ADDI R5, R4, 3          ; RAW hazard on R4
70000000 // PC 5: HALT                    ; stop execution
    
```

10.3 Simulation Results

This section presents the functional verification results of the pipelined predicated RISC processor. Each test program was loaded individually into `program.hex`, simulated using EDA Playground with EPWave, and validated by inspecting pipeline waveforms across all stages (IF, ID, EX, MEM, WB). For each test, the waveform confirms correct instruction flow, hazard handling, control behavior, and writeback correctness.

10.3.1 Test 1: Logical R-Type Instructions

Test file: test_logical_rtype.hex

This test verifies correct execution of logical R-type instructions (OR, AND, NOR) and proper register writeback. The waveforms confirm correct ALU outputs, destination register propagation through the pipeline, and final writeback in the WB stage.

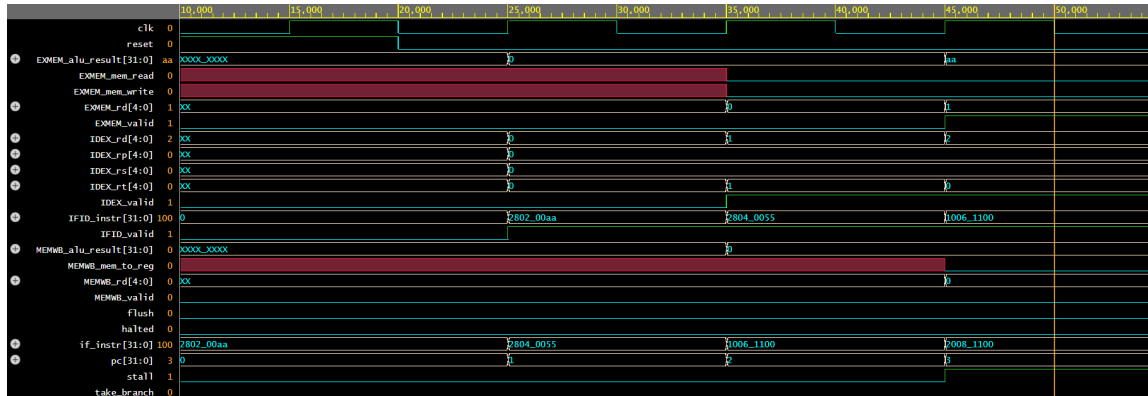


Figure 4: Pipeline waveform for logical R-type instructions

Waveform Analysis The waveform shows uninterrupted pipeline flow with no stall or flush events. The ALU result is computed in the EX stage and propagated through the EX/MEM and MEM/WB registers. The EXMEM_rd and MEMWB_rd signals correctly identify the destination register, while MEMWB_reg_write is asserted during the WB stage, confirming correct writeback behavior. No hazards are detected, and stall remains low throughout execution.

10.3.2 Test 2: Load-Use Hazard Handling

Test file: test_load_use_hazard.hex

This test validates correct detection and handling of load-use hazards. A stall is introduced when a dependent instruction follows a load. The waveform shows assertion of the stall signal, insertion of a bubble, and correct data forwarding once the load completes.

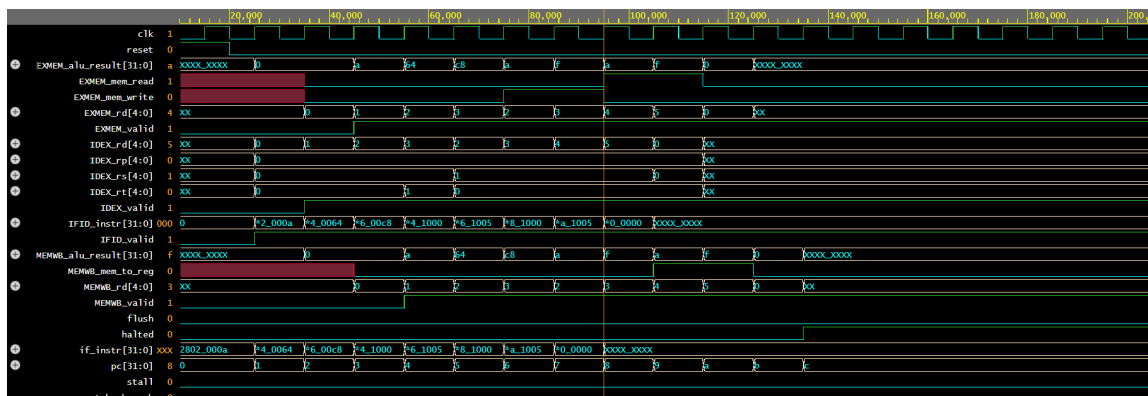


Figure 5: Load-use hazard stall and pipeline recovery

Waveform Analysis A load-use hazard is correctly detected when an instruction in the ID stage depends on the result of a load in the EX stage. This condition is indicated by `IDEX_mem_read = 1` and matching destination and source registers. The `stall` signal is asserted for one cycle, freezing the PC and IF/ID registers while inserting a bubble into the pipeline. After the load completes, normal execution resumes with correct data forwarding.

10.3.3 Test 3: Consecutive Data Hazards

Test file: `test_consecutive_hazards.hex`

This test stresses the forwarding and hazard detection logic using multiple dependent instructions in sequence. The waveform confirms correct forwarding from EX/MEM and MEM/WB stages without unnecessary stalls.

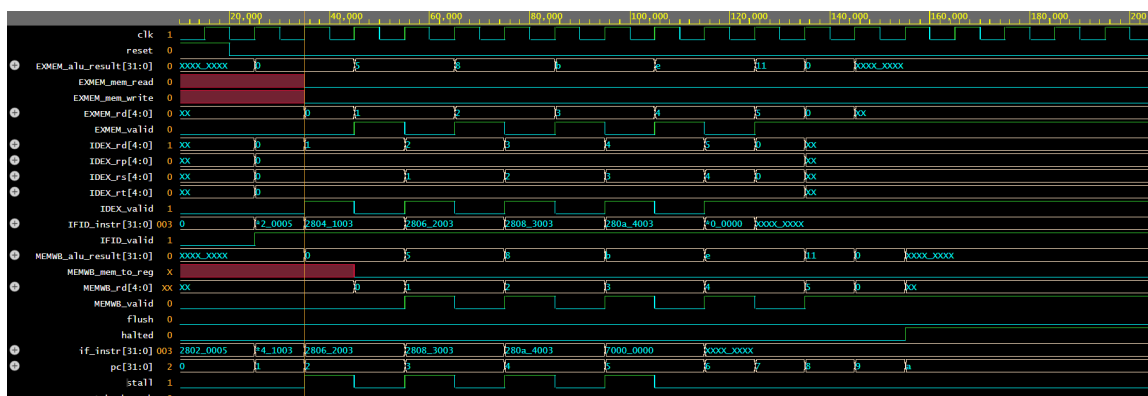


Figure 6: Forwarding across consecutive dependent instructions

Waveform Analysis The waveform demonstrates correct forwarding across multiple consecutive RAW dependencies. Values are forwarded directly from the EX/MEM and MEM/WB stages, as indicated by stable ALU results and continuous assertion of valid signals. No stalls are introduced, confirming correct priority-based forwarding logic.

10.3.4 Test 4: Jump Instruction

Test file: `test_jump.hex`

This test verifies correct jump behavior. When a jump instruction is executed, the PC is updated with the target address, and the pipeline is flushed. The waveform shows assertion of the `flush` signal and correct PC redirection.

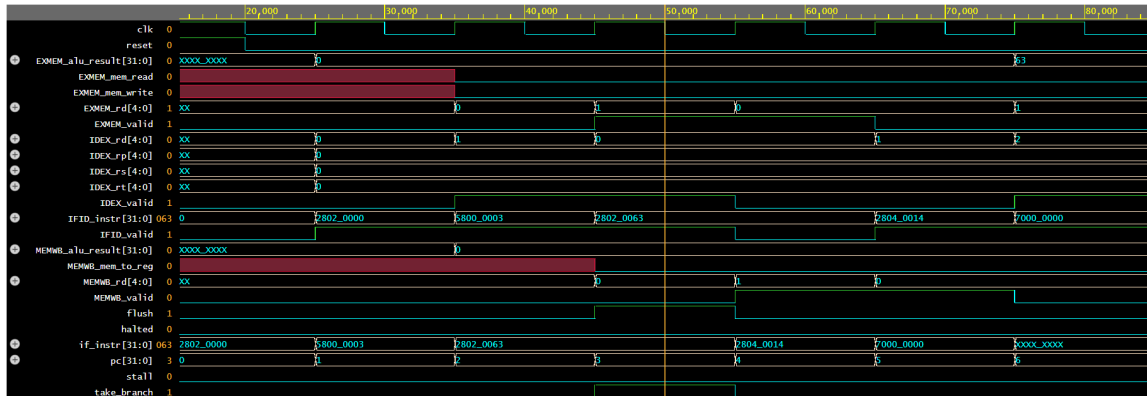


Figure 7: Pipeline flush and PC update during jump instruction

Waveform Analysis Upon execution of the jump instruction in the EX stage, the `take_branch` signal is asserted and the `flush` signal invalidates the IF/ID and ID/EX pipeline registers. The PC is updated with the jump target, and instruction fetch resumes from the new address. Instructions in the wrong path are successfully discarded.

10.3.5 Test 5: CALL Instruction

Test file: `test_call.hex`

This test validates the `CALL` instruction functionality. The return address is correctly written to the link register while control flow is redirected to the target address. The waveform confirms correct ALU result propagation and register writeback.

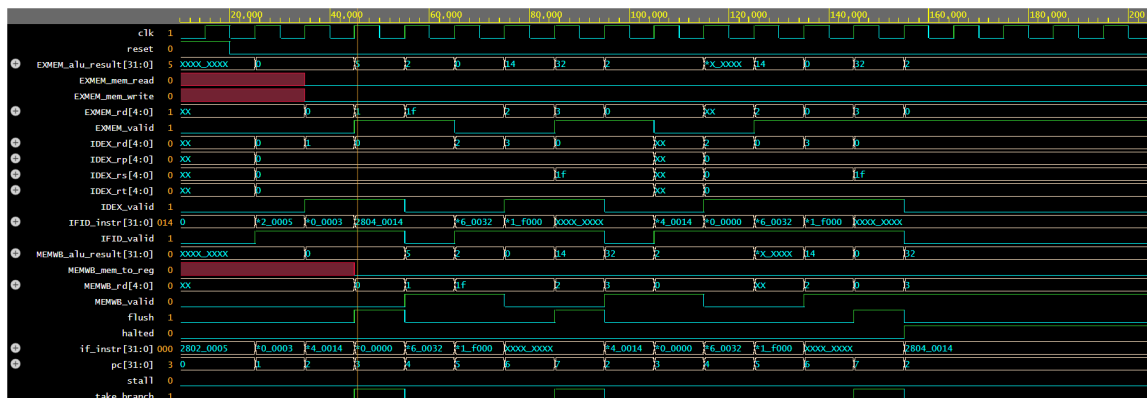


Figure 8: CALL instruction execution and return address handling

Waveform Analysis The `CALL` instruction updates the program counter and writes the return address (PC+1) to register R31. This behavior is confirmed by the ALU result, destination register selection, and assertion of `MEMWB_reg_write`. Pipeline flushing occurs as expected, and execution resumes at the call target.

10.3.6 Test 6: Memory Access with Offset

Test file: `test_memory_offset.hex`

This test verifies correct address calculation and memory access using offsets for `LW` and `SW` instructions. The waveform shows correct ALU address computation, memory read/write control signals, and correct writeback behavior.



Figure 9: Load and store operations with address offsets

Waveform Analysis The ALU correctly computes memory addresses by adding the base register and sign-extended immediate offset. The waveform shows proper assertion of EXMEM_mem_write and EXMEM_mem_read during store and load operations respectively. Data is correctly written to and read from memory, followed by proper writeback.

10.3.7 Test 7: Predicated Memory Instructions

Test file: test_predicated_memory.hex

This test validates predicated execution of memory instructions. Memory operations are executed or suppressed based on the predicate register. The waveform confirms correct evaluation of the predicate and conditional memory access.

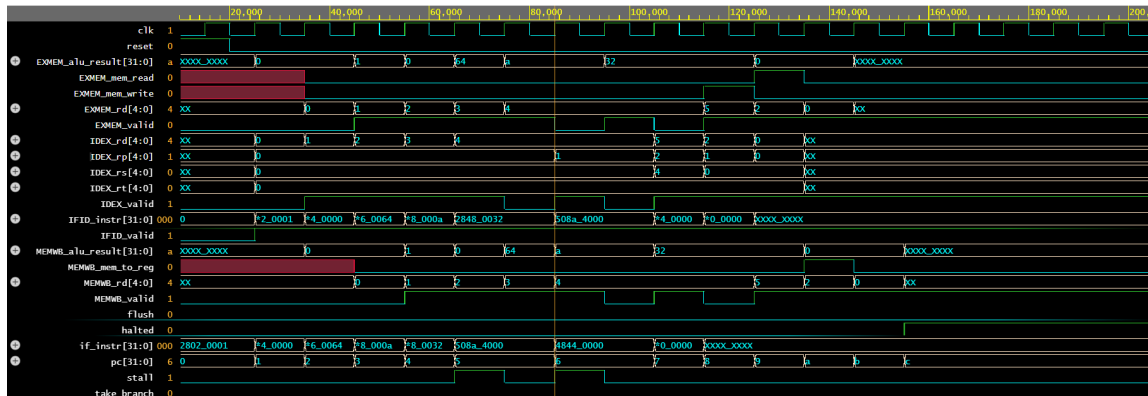


Figure 10: Conditional memory access using predicate registers

Waveform Analysis Predicate evaluation occurs in the EX stage. When the predicate condition fails, the predicate_pass signal is deasserted, preventing memory writes and register updates despite the instruction flowing through the pipeline. When the predicate condition holds, normal execution resumes. This confirms correct predicate gating without introducing stalls.

Overall, the simulation results confirm correct functionality of the pipelined predicated RISC processor across arithmetic, control flow, memory access, hazard handling, and predicated execution.

10.4 Test Results Summary

Test Name	Status	Notes
Arithmetic R-Type	PASS	All operations correct
Logical Operations	PASS	OR, AND, NOR verified
Load-Use Hazard	PASS	Stall inserted correctly
Memory Offset	PASS	Offset addressing works
Jump Instruction	PASS	PC updated correctly
CALL and JR	PASS	Function call/return works
Predicated Execution	PASS	Predication logic correct
Consecutive Hazards	PASS	Forwarding successful

Table 5: Test Results Summary

11 Performance Analysis

Table 6 summarizes the measured performance metrics obtained from simulation. Cycle counts and stall events were extracted directly from the testbench output.

Test Case	Instr. Count	Cycles	Stalls	CPI
Arithmetic / Logical R-Type	6	10	1	1.67
Load-Use Hazard	6	11	2	1.83
Memory Offset Addressing	8	11	0	1.38
Jump Instruction	6	9	0	1.50
CALL / JR	6	13	0	2.17
Predicated Execution	8	13	2	1.63
Consecutive RAW Hazards	6	13	4	2.17

Table 6: Measured performance statistics from simulation

12 Code Organization

12.1 File Structure

```

project/
  rtl/
    design.sv
testbench/
  testbench.sv          ← main (all tests, checking)
  waveform_testbench.sv ← single-test waveform TB each run
  test_programs/
    test_call.hex
    test_consecutive_hazards.hex
    test_jump.hex
    test_load_use_hazard.hex
    test_logical_rtype.hex
    
```

```

test_memory_offset.hex
test_predicated_memory.hex
docs/
report.pdf
datapath.png

```

13 Team Member Contributions

13.1 Work Distribution

Team Member	Responsibilities
Asmaa Fares	• Datapath design and implementation • ALU and register file modules • Pipeline register design • Report: Sections 4, 5
Aya Fares	• Control unit design and implementation • Hazard detection and forwarding logic • Predication logic • Report: Sections 3, 6
Baker Shalabi	• Testbench development • Test program creation • Simulation and debugging • Report: Sections 7, 8
<i>All members participated in design discussions, integration, and final testing</i>	

Table 7: Team Member Contributions

14 Challenges and Solutions

14.1 Challenge 1: Load-Use Hazard Detection

Problem: Initially, forwarding logic attempted to forward load data before it was available from memory.

Solution: Implemented hazard detection unit that specifically checks for load instructions in EX stage and stalls the pipeline for one cycle, allowing the load data to reach MEM stage before being forwarded.

14.2 Challenge 2: Predicate Register Forwarding

Problem: Predicate registers (Rp) also need forwarding, which was initially overlooked.

Solution: Extended forwarding logic to include all four register reads (Rs, Rt, Rp, Rd), ensuring predicate values are always current.

14.3 Challenge 3: CALL Instruction Return Address

Problem: Determining the correct return address (PC+1) timing.

Solution: Store PC+1 in the EX stage when CALL is detected, writing to R31 through the normal writeback path.

14.4 Challenge 4: Special Register Management

Problem: Ensuring R0, R30, R31 maintain correct values across the pipeline.

Solution: Explicitly prevent writes to R0 and R30 in WB stage, and continuously update R30 with PC value each cycle.

15 Conclusion

This project successfully designed and implemented a 32-bit predicated RISC processor with a 5-stage pipeline. The processor supports 15 instructions covering arithmetic, logical, memory access, and control flow operations, with full predicated execution capability.

15.1 Key Achievements

- Complete datapath and control path implementation
- Functional hazard detection and forwarding
- Successful predicated execution mechanism
- Comprehensive verification with 7 test programs
- All specified instructions working correctly

15.2 Learning Outcomes

- Deep understanding of pipeline architecture and hazards
- Practical experience with RTL design and verification
- Insight into predicated execution benefits and costs
- Hands-on experience with Verilog and simulation tools

16 AI Tools Disclosure

This project used AI-assisted tools during development.

- **Tools Used:** ChatGPT
- **Purpose:**
 - Clarification of pipeline and hazard concepts
 - Assistance with LaTeX formatting and diagram structuring
 - Verification report wording and organization
- **Extent:** AI tools were used for guidance and clarification only. All RTL code, testbench implementation, and architectural decisions were fully understood and written by the project team.